Individual Project Contribution Report

Student Name: Jc Gil Balicao

Group Name: MiniOS Alpha

Assigned Role: Process and Memory Developer

1. **What did I contribute?**

developing the process management and memory management

2. **What features did I personally develop?**

Process management - processes.py

Memory management - memory.py

3. **What problems did I encounter?**

One challenge was making sure that process and memory operations stayed within the simulator's limits.

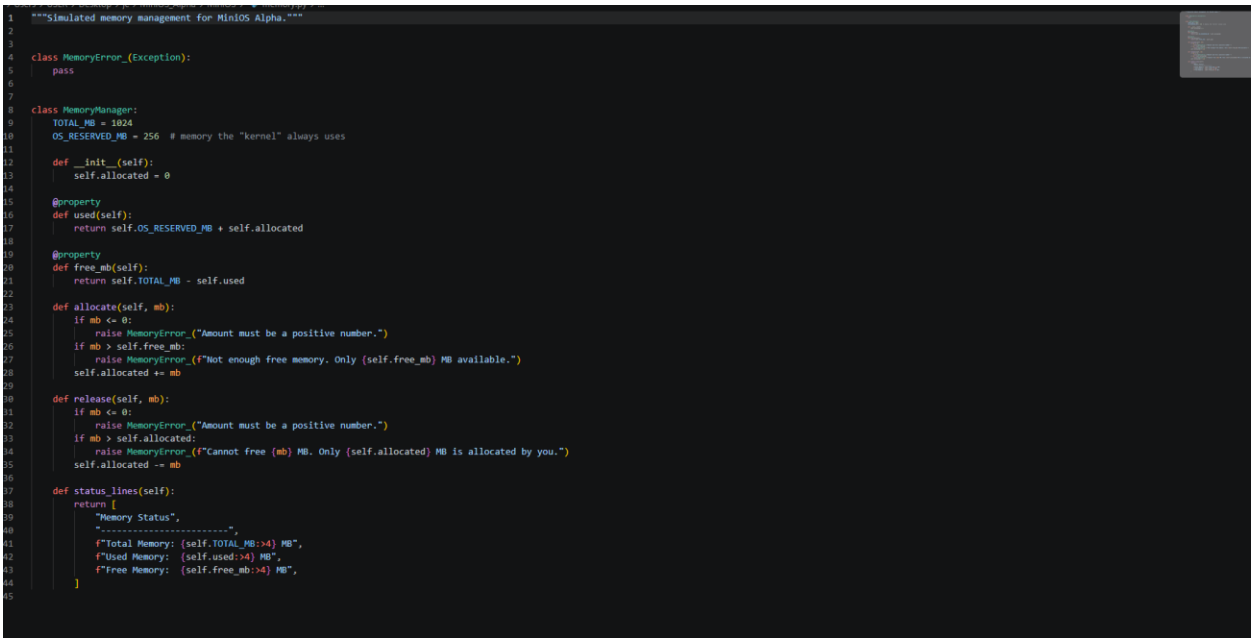
4. **How did I solve those problems?**

I used validation checks before changing the process table or memory totals. The process manager checks the active-process limit and verifies that a PID exists before terminating it.

5. **What did I learn from this project?**

I learned how process management and memory management can be shown in an operating system simulator. I also learned the importance of process management, memory management tracking resources carefully validating user input carefully handling errors carefully and separating program features into modules.

6. **Evidence of my contribution**



```python
"""Simulated memory management for MiniOS Alpha."""


class MemoryError_(Exception):
    pass


class MemoryManager:
    TOTAL_MB = 1024
    OS_RESERVED_MB = 256  # memory the "kernel" always uses

    def __init__(self):
        self.allocated = 0

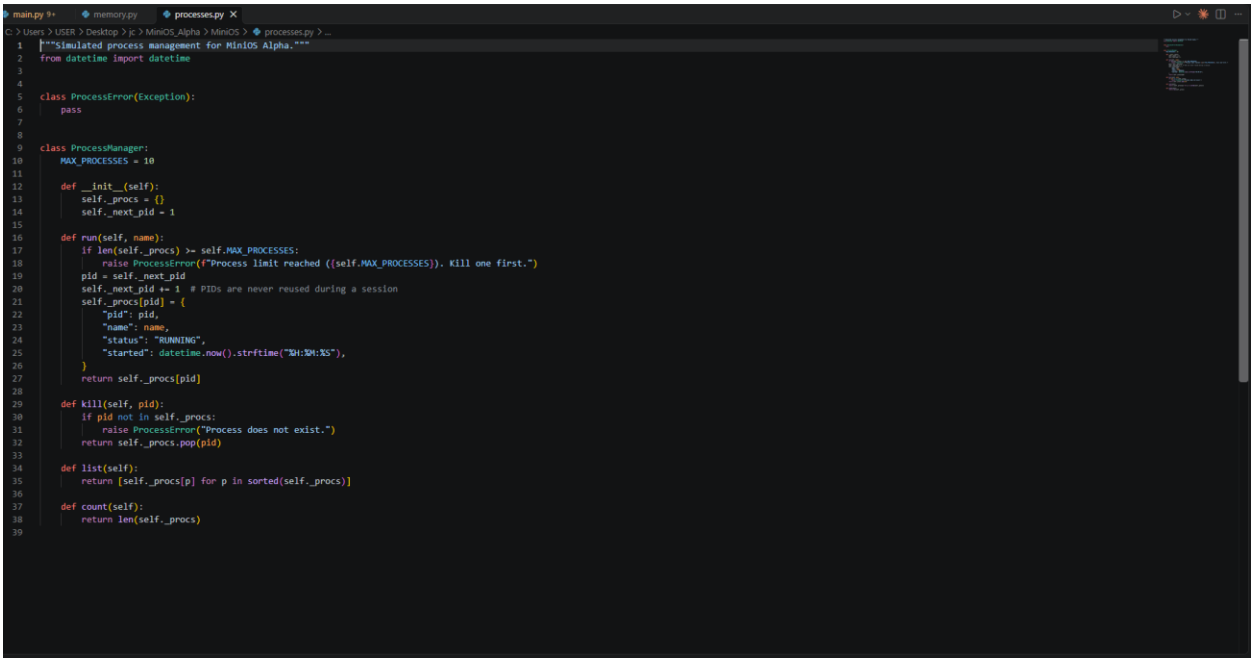
    @property
    def used(self):
        return self.OS_RESERVED_MB + self.allocated

    @property
    def free_mb(self):
        return self.TOTAL_MB - self.used

    def allocate(self, mb):
        if mb <= 0:
            raise MemoryError_("Amount must be a positive number.")
        if mb > self.free_mb:
            raise MemoryError_(f"Not enough free memory. Only {self.free_mb} MB available.")
        self.allocated += mb

    def release(self, mb):
        if mb <= 0:
            raise MemoryError_("Amount must be a positive number.")
        if mb > self.allocated:
            raise MemoryError_(f"Cannot free {mb} MB. Only {self.allocated} MB is allocated by you.")
        self.allocated -= mb

    def status_lines(self):
        return [
            "Memory Status",
            "------------------------",
            f"Total Memory: {self.TOTAL_MB:>4} MB",
            f"Used Memory:  {self.used:>4} MB",
            f"Free Memory:  {self.free_mb:>4} MB",
        ]
```



main.py 9+ memory.py processes.py

C: > Users > USER > Desktop > jc > MiniOS_Alpha > MiniOS > processes.py > ...

```python
"""Simulated process management for MiniOS Alpha."""
from datetime import datetime


class ProcessError(Exception):
    pass


class ProcessManager:
    MAX_PROCESSES = 10

    def __init__(self):
        self._procs = {}
        self._next_pid = 1

    def run(self, name):
        if len(self._procs) >= self.MAX_PROCESSES:
            raise ProcessError(f"Process limit reached ({self.MAX_PROCESSES}). Kill one first.")
        pid = self._next_pid
        self._next_pid += 1  # PIDs are never reused during a session
        self._procs[pid] = {
            "pid": pid,
            "name": name,
            "status": "RUNNING",
            "started": datetime.now().strftime("%H:%M:%S"),
        }
        return self._procs[pid]

    def kill(self, pid):
        if pid not in self._procs:
            raise ProcessError("Process does not exist.")
        return self._procs.pop(pid)

    def list(self):
        return [self._procs[p] for p in sorted(self._procs)]

    def count(self):
        return len(self._procs)
```